Nivaas — Technical Design Document

GitHub: {{GITHUB_URL}} **Live app:** {{LIVE_URL}}

Submitted by Debanshu Devgoswami. Everything labelled "Assumption" is my own call where the brief was silent, made so the build could proceed — each one is listed in §10.

---

1. The product and the problem

Company A is a facility-management company running a gated township in Pune: 2,400 apartments in 28 towers, about 180 staff on site, and ₹22–26 crore of maintenance billing a year. Today the work runs on Excel bills emailed to residents, payments in Tally matched by hand, complaints arriving by phone and 28 tower WhatsApp groups, and paper visitor registers at four gates.

The brief quantifies what that costs. At 2,400 flats × ₹7,500 a month, billing is **₹21.6 crore a year**, and:

- **3% is never collected — ₹64.8 lakh a year.** Bills go out late, residents can't see what they owe or pay in two taps, and nothing follows up systematically.
- **Matching labour — ₹8.64 lakh a year.** Three accounts staff spend 60% of their time matching cheque/transfer/UPI payments to flats by hand before entering them in Tally.
- **Reopened complaints — ₹10.13 lakh a year.** Of 1,800 complaints a month, 25% (450) are reopened; at 45 minutes and ₹250/hour each, poor first-time resolution and invisible status burn real money.

Total problem: ₹83.57 lakh a year. The client's success criteria over six months: on-time payments from 70% to 90%, payment-matching time down 80%, reopened complaints from 25% to under 8%.

My read of those numbers drove every scope decision below: the two money flows (collecting and matching) and the complaint flow (status visibility and first-time-right filing) are where the ₹83.57 lakh sits, so they are what I built end to end.

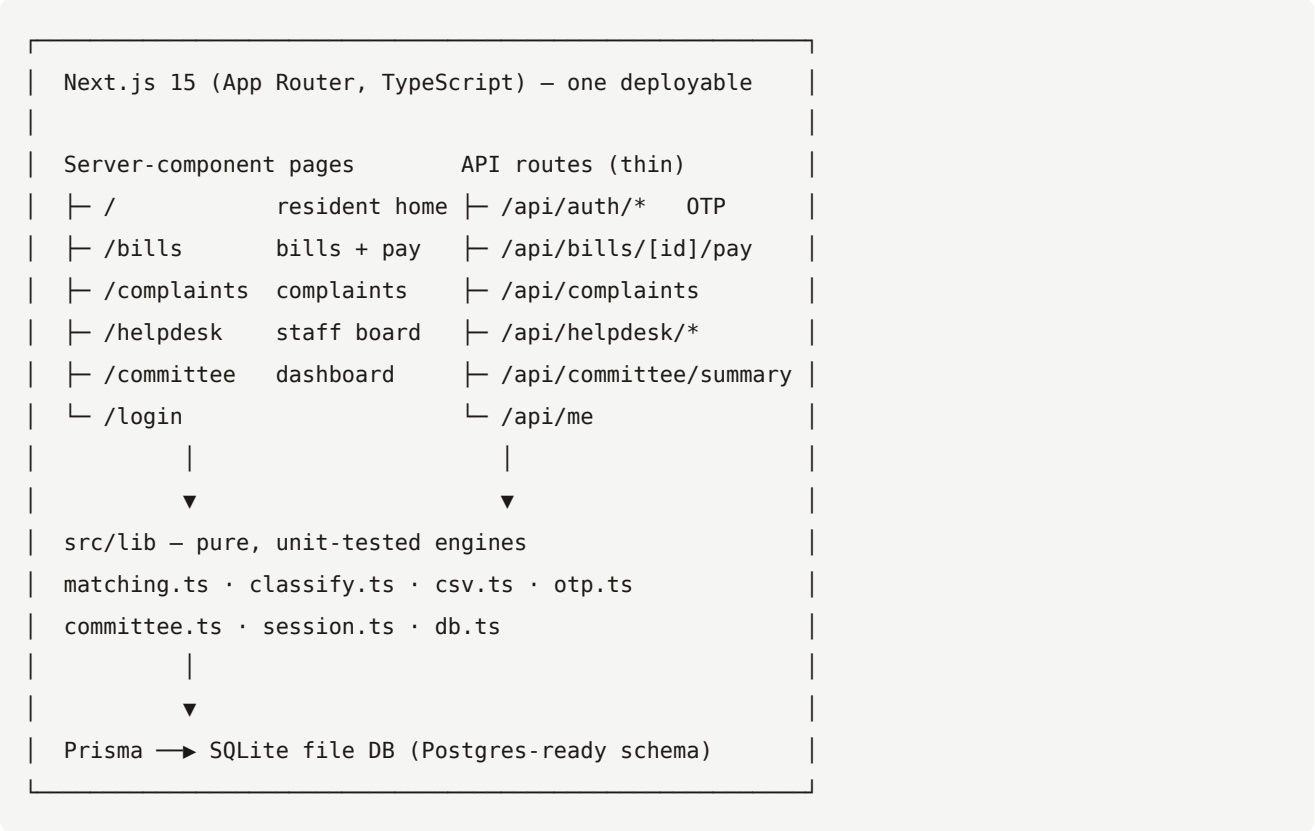
2. What I built

The core slice from the brief, working end to end on seeded data:

- **OTP login** linked to a flat, with three roles — resident, help-desk staff, committee member — each landing on their own view.
- **Maintenance bills with test payment**: bills computed from flat area × the society rate, late fees, a mock payment sheet (UPI/Card, test mode) that returns a receipt, and a per-flat payment ledger.
- **Bank statement matching**: upload or paste a statement CSV; the engine posts rows it can prove and queues the rest for a human. This is the hard part of the brief, and §6 is devoted to it.
- **Complaints with assignment and status**: a resident raises a complaint (photo optional); AI classifies category and urgency, auto-assigns the trade team, links duplicates, and the resident always sees where it stands. The help desk works a board with one-tap category correction, team and status controls.
- **Committee view**: collection percentage, total outstanding, dues ageing buckets and complaint health, read from the same ledger the other roles write to.

Notices are included (they appear on the resident home). The rest of the brief's MUST HAVE list — visitor/gate management, amenity booking, family/tenant member management — is deliberately *not* in this build; §11 says why and in what order I would add them.

3. Architecture, and why



One Next.js monolith. Pages (server components), API routes and the domain engines live in a single codebase and a single deployable. For a solo build with three role-based views over one ledger, separate front/back repos would double the deployment and type-drift risk for no benefit; the client is one facility company, not a platform team. Server components read the ledger directly through Prisma; the API routes exist where the browser must act (pay, raise, match, patch).

Engines are pure functions; routes stay thin. `matching.ts`, `classify.ts`, `csv.ts` and `otp.ts` take plain data and return plain data — no database, no I/O. Every API route follows the same shape: authenticate → validate input → call an engine → write in one transaction. That is what makes the risky logic (money matching, safety escalation) exactly unit-testable, and it is where most of the 41 automated tests point.

Prisma + SQLite. The schema is the single source of truth, and SQLite gives a real relational database — transactions, unique constraints, foreign keys — with zero infrastructure to run, seed or pay for. The schema uses no SQLite-specific features; moving to Postgres is a provider line plus `DATABASE_URL`. For 2,400 flats and a few hundred transactions a day, SQLite is honestly sufficient for v1; I would still move to Postgres before a real rollout for concurrent writes from four gate tablets (§11).

Money is integer rupees. Every amount column is an `Int` of whole rupees — society billing (rate × sq ft, rounded late fees) is always whole-rupee here, and integer equality is what makes the matching engine's “amount equals the due exactly” test trustworthy. No floats ever touch the ledger.

Mocked boundaries, real seams. Per the brief, payments, OTP delivery and the bank feed are mocked — but each sits behind the same interface a real provider would use: the payment sheet returns a receipt object a real gateway callback would produce; the OTP is a real code, generated, hashed, expired and attempt-capped, only its *delivery* is a console log instead of an SMS; the bank feed is a CSV upload today, an API pull tomorrow. The mocks are at the edges, never in the domain logic.

Sessions. A signed JWT (jose, HS256) in an httpOnly cookie, 7-day expiry, carrying user id, name and role. Every page and API route checks the session and the role server-side (§9).

4. Data model

Eight tables (`prisma/schema.prisma` is authoritative):

Table	Holds	Key fields / notes
`User`	People who can sign in	`phone` unique (login identity), `role` ∈ RESIDENT / HELPDESK / COMMITTEE
`Flat`	One apartment	`code` unique (“T07-1203”), `tower`, `areaSqft`, `ownerName`, `upiId` (matching evidence)
`Membership`	Who belongs to which flat	`type` ∈ OWNER / TENANT / FAMILY; unique (user, flat)
`Bill`	One flat, one month	`baseAmount` = rate × area, `lateFee`, `paidAmount`, `status` ∈ UNPAID / PARTIALLY_PAID / PAID; unique (flat, month)
`Payment`	Money in, from any source	`source` ∈ ONLINE / BANK_MATCH / MANUAL, `status` ∈ POSTED / QUEUED / REJECTED, nullable `flatId` / `billId`, `reference`, `payerUpiId`, `rawRow` (JSON)
`Complaint`	One complaint	`category`, `urgency`, `status`, `assignedTeam`, `aiConfidence`, `rating`, `resolvedAt`, **self-relation** `duplicateOfId` → `Complaint`
`Notice`	Township notices	title, body, timestamp
`OtpCode`	One live OTP per phone	`codeHash` (SHA-256, never the code), `attempts`, `expiresAt`

Three modelling decisions worth calling out:

- **A queued payment is a `Payment` row with `status = QUEUED` and no flat.** The original bank row is preserved verbatim in `rawRow`, so nothing the bank said is ever lost, and resolving the queue is a status change plus a flat link — the audit trail survives. When a human posts it to a flat, it becomes a normal POSTED payment against that flat's oldest open bill.
- **Duplicates are a self-relation, not a merge.** A duplicate complaint keeps its own row, its own status and its own resident; `duplicateOfId` only *links* it to the first report. Ten flats reporting one lift failure is severity signal — the data model refuses to delete it (§7).
- **`Complaint.aiConfidence` is stored at raise time.** What the AI believed when it filed the complaint stays on the record even after the help desk corrects the category, so the classifier's accuracy can be audited later against human corrections.

5. Main flows

5.1 Billing and payment

Bills are generated per flat as `base = areaSqft × ₹6.50` for the month, with a due date; an overdue bill accrues a late fee (Assumption A1). The resident home leads with the total due and a per-bill breakdown — including the late fee, itemised, so the number is never a surprise. Paying opens a test-mode sheet (UPI or Card); the API refuses to pay a bill that is already PAID (409) and refuses to pay another flat's bill (403). A successful payment writes a `Payment` (source ONLINE, status POSTED) and updates the bill's `paidAmount` and `status` in the same transaction, then returns a receipt with a `MOCKPAY-…` reference. The bills page is the ledger: every bill with its status, and every posted payment with its source (“Paid in app” / “Bank transfer (matched)” / “Recorded by help desk”), so a resident can reconcile their flat's money without calling anyone.

5.2 Bank statement matching

Covered in §6 — it is the hard part and the heart of the build.

5.3 Complaints

A resident picks their flat, writes a title and a line or two of detail, optionally attaches a photo (images only, ≤ 5 MB, stored under `public/uploads/`). On submit, the complaint passes through the classifier (§7): category, urgency, confidence, duplicate check, auto-assignment to the trade team for the category (Plumbing, Electrical, Lift Maintenance, Carpentry, Housekeeping, Security; OTHER goes to a General pool and starts OPEN). The resident immediately sees the verdict — including, when confidence is low, an honest “the system wasn't sure; the help desk will confirm” note, and, when a safety rule fired, the escalation notice. Status then moves OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED, with resolution time recorded (`resolvedAt`) for the committee's average; the resident rates the work on closure (the field and display exist; rating is seeded for closed complaints — residents rating from the UI is a small follow-up I cut, §11).

The help desk board lists every complaint with flat, age, urgency, duplicate badges and the AI's confidence. Category correction is a single `<select>` change — one PATCH, one field — because the brief is explicit that the help desk “can change a category in one tap”, and a wrong category must never slow the resident down.

5.4 Committee view

One server-side aggregation (`src/lib/committee.ts`) over the same ledger: the current month's billed vs collected and collection %, total outstanding across all open bills, dues ageing buckets (0–30 / 31–60 / 61–90 / 90+ days, aged by each flat's *oldest* unpaid bill — the conservative view a committee actually wants), open complaints by status and category, and average resolution hours. No hand-built Excel, and nothing on this screen can disagree with what residents and staff see, because it is the same rows.

6. The hard part: payment matching

The society receives one bank statement for the whole township. Each row must be credited to exactly one flat's bill — or not credited at all. A wrong credit is worse than a slow one: it tells one resident they paid when

they didn't, and leaves another marked a defaulter. So the engine's rule, stated once and enforced everywhere:

> **A row is auto-posted only when the evidence identifies one flat beyond doubt. Anything weaker goes to the accounts queue for a human. An amount alone never auto-posts** — in a township of 2,400 flats, dozens of bills share an amount every month.

Each row is tested against six tiers, strongest evidence first (`src/lib/matching.ts`):

Tier	Evidence	Outcome
1	The row's reference/UTR exactly matches a reference registered to a bill (e.g. quoted from a payment link)	**POSTED** to that bill
2	Payer UPI id belongs to a flat **and** the amount equals that flat's full remaining due	**POSTED**
3	Payer UPI id belongs to a flat and the amount is *less* than the due	**POSTED** as a part-payment (bill → PARTIALLY_PAID)
4	The narration names exactly one flat code (e.g. “T07-1404”) **and** the amount equals its due exactly	**POSTED**
5	The amount equals exactly one open bill's due, but there is no other evidence	**QUEUED** — amount alone never posts
6	Anything else: ambiguous amount (two+ flats owe it), overpayment against a UPI flat, UPI flat with no open bill, unknown payer	**QUEUED**

Details that matter:

- **Per-bill remaining-due tracking inside a batch.** The engine decrements a bill's remaining due as rows post against it within one statement, so two rows in the same CSV can never both credit the same bill in full; a second row against a settled bill queues with the reason spelled out.
- **Overpayments queue whole.** An overpayment is never truncated to the due and silently absorbed — the full amount waits for a human, because the excess belongs to someone.
- **The parser is tolerant, the engine is not.** `src/lib/csv.ts` accepts header aliases, quoted cells and `6,466.00`-style amounts; a row it cannot parse is queued as an explicit UNPARSED entry with its raw text preserved — never dropped silently.
- **Every verdict carries its reason in plain English** (“UPI id belongs to T12-0408 and amount equals the full due”, “Amount equals the due of T18-0902 only, but an amount alone never auto-posts”), and the help-desk UI shows tier + reason per row. An accountant reviewing the run can see *why* each rupee went where it went.
- **Applying a run is one transaction.** POSTED verdicts write payments and update bills; QUEUED verdicts write queue rows with `rawRow` preserved. A failed run changes nothing.

The bundled `public/sample-bank-statement.csv` is crafted against the seed data to exercise the tiers — Swati's October bill posts by UPI + amount (tier 2), a narration flat-code match posts (tier 4), a part payment from Aamir's UPI posts (tier 3), while an ambiguous ₹7,150 that two flats owe, a unique amount-only ₹8,125, and an unknown ₹4,321 cheque all queue (tiers 6, 5, 6). An integration test asserts exactly this 3-posted / 3-queued mix, so the demo cannot silently rot if the seed changes.

Against the brief's math: this flow is the ₹8.64 lakh matching-labour line. The engine doesn't claim 100% automation — it claims the provable majority posts itself, and the human's queue contains only genuinely ambiguous money, each with its evidence attached.

7. The AI part: classify, escalate, link — and be honest when unsure

`src/lib/classify.ts` runs three steps, in a strict order that is the whole design:

Step 1 — safety rules, before any scoring. A list of phrases — *fire, gas leak / smell of gas, lift stuck / trapped in, electric shock, short circuit, sparking, burst pipe, flooding* — is checked first. A hit forces urgency **CRITICAL**, marks the complaint `escalated` (surfaced in the UI as “escalated to the duty manager”), and assigns the owning trade's category where one exists (lift-stuck → LIFT, shock → ELECTRICAL, burst pipe → PLUMBING; fire and gas map to OTHER because no trade owns them — the duty manager is the first responder). The brief's wording is implemented literally: safety words escalate *by rule, whatever the AI says* — so the rule is not part of the classifier's scoring and cannot be outvoted by it. In the data, escalation is *derived* from CRITICAL urgency rather than stored as a separate flag, so it can never be silently un-flagged.

Step 2 — weighted keyword classification. Each category owns a keyword table with weights: 2 for distinctive words (*leakage, flush, mcb, hinge*), 1 for suggestive shared words (*water, door, light*). Scores are summed per category; the winner's share of the evidence becomes a confidence in 0..1. Urgency is derived from the category result plus severity words. The resident sees category, urgency and confidence immediately after raising the complaint.

Step 3 — when it is wrong or unsure, the system says so. If the best score is too weak, the complaint files as **OTHER with confidence < 0.5**, the resident is told the help desk will confirm the category, and the help desk corrects it in one tap (one field, one PATCH). The complaint is never blocked, delayed or hidden because the AI was unsure — the human path is the default, the AI is an accelerator on top of it.

Duplicate linking (TF-IDF cosine). On raise, the complaint text (title + description) is compared against still-open complaints from the **same tower** created within **72 hours**, using TF-IDF vectors and cosine similarity — the same technique I used in an earlier project (YC Match) for matching text against 6,295 company descriptions. Similarity ≥ **0.45** links the new complaint to the earliest matching one via `duplicateOfId`. Linked, **never closed**: the board shows “Duplicate of …” on the copy and “+N linked” on the original, so ten flats reporting the same lift reads as one incident with ten witnesses — severity information a facility manager wants, not noise. The seed contains a four-complaint lift cluster in T12 that exercises exactly this.

Honest limits. This is a keyword classifier, and I chose it deliberately for v1: it is deterministic, explainable in an interview and in front of a client (“these words, these weights”), needs no API key, no network and no training data, and its failure mode (OTHER + human confirm) is safe. It will miss paraphrases a language model would catch (“the thing from which water comes is broken”), and its confidence is a heuristic share-of-evidence, not a calibrated probability. The seam is the point: `classifyComplaint(text) → {category, urgency, confidence, escalated}` is the only contract callers know, so an LLM classifier can replace the internals later without touching the API route, the board or the resident UI (§11). Anyone claiming more for a v1 classifier on day one is selling, not engineering.

8. UI and design decisions

The client asked for “good, minimal design… nothing on screen that lacks a purpose”, and residents who open the app “for seconds at a time”. The design system follows from that:

- **One accent, restrained palette.** Deep teal `#0F766E` for brand and primary actions; warm stone neutrals for everything else; colour is spent only where it carries meaning — status.
- **One chip system everywhere** (`src/components/chips.tsx`). Bill PAID green / PARTIALLY_PAID amber / UNPAID rose; complaint OPEN blue / ASSIGNED indigo / IN_PROGRESS amber / RESOLVED green / CLOSED gray; payment POSTED green / QUEUED amber; and CRITICAL urgency is a **solid red chip** — the one place the system shouts, because a safety escalation should read as an alarm from across a room. A colour learned on one screen means the same thing on all of them.
- **Home is a glance, not a dashboard.** Total due first with the per-bill breakdown and one Pay action, then open complaints, then notices — the brief's home-screen contents in the brief's order. Every primary action is within two taps of opening the app (Home → Pay; Home → raise complaint).
- **Resident targets are ≥ 44 px** (bottom nav, pay buttons, form submits, demo-login chips), with a bottom tab bar on mobile (with a clear active state) and a top nav on wider screens. **Staff screens optimise the other way**: the help-desk board is a dense table with a sticky tab bar, because staff scan a hundred rows, not four.
- **Cards are consistent** — white, `rounded-2xl`, one stone ring, 16–20 px padding — on a `max-w-5xl` column for app screens and `max-w-md` for login, so the eye never re-learns the page.
- **No dead ends and no blank screens.** Every main route has a skeleton `loading.tsx`; lists have written empty states (“No open complaints. If something needs fixing, raise one…”) rather than voids; client fetches surface an error with a retry button; a root error boundary catches page failures with a way back. The AI's uncertainty is also UI: low-confidence filings and safety escalations are shown to the resident as plain-language cards, not hidden in a log.

9. Security and privacy

- **OTP done properly, delivery mocked.** Codes are 6-digit, generated per request, stored only as SHA-256 hashes in `OtpCode`, expire in 5 minutes, one live code per phone, verify attempts capped at 5. The demo shortcut (`123456`) is documented in the README and deployment guide — it exists so a reviewer can sign in; it is an explicit demo affordance, not a hidden back door, and removing it for production is one constant.
- **Sessions** are signed (HS256) httpOnly cookies; the role travels in the token and is **enforced server-side on every page and API route** — a resident calling the help-desk APIs gets a real 403, the committee summary is unreachable without the COMMITTEE role, and paying another flat's bill is refused (403), as is re-paying a settled bill (409). There is no client-side-only “security”.
- **Input validation** at the route boundary: phone and code shapes, payment method enum, category/status/team enums on the board PATCH, complaint field lengths, photo uploads restricted to images ≤ 5 MB. Money writes are transactional.
- **No secrets in the repository.** `.env` is git-ignored; the committed `.env.example` and `deployment.md` carry names and purposes only. `SESSION_SECRET` has a clearly-marked dev-only fallback so a fresh clone runs with zero setup; production must set it (Render generates it from the blueprint).

- **DPDP Act 2023.** The build holds phone numbers (login identity) and complaint photos — personal data, handled minimally: phones are never displayed to other roles, photos only to the owning resident and staff. The visitor module — the brief's heaviest DPDP surface — is in §11; its 90-day photo deletion and gate notice are designed requirements there, not afterthoughts.

10. Tests

41 automated tests, all green (`npm test`, Vitest):

- `matching.test.ts` — every tier: reference match, UPI full and part payments, narration flat-code match; amount-only → QUEUED; ambiguous amount → QUEUED; overpayment queued whole; settled-bill double-credit prevented inside one batch.
- `classify.test.ts` — safety phrases force CRITICAL + escalation (including when embedded in otherwise-benign text), category scoring for each trade, weak text → OTHER with low confidence, and TF-IDF duplicate detection for a same-tower lift cluster (and non-linking for different towers / stale reports).
- `otp.test.ts` — issue/verify happy path, wrong code, expiry, attempt cap, demo code behaviour.
- `csv.test.ts` — header aliases, quoted cells, Indian-format amounts, unparseable rows surfaced rather than dropped.
- `matching.integration.test.ts` — the bundled sample statement against the seeded bills asserts the exact 3-posted / 3-queued tier mix the reviewer will see.

Beyond the unit suite, I ran a **17-step smoke test against the production build** (`next start`): both resident logins, help-desk and committee logins, bill payment + receipt + refused re-payment, a safety-word complaint filing as CRITICAL, the sample CSV run (3 POSTED / 3 QUEUED), queue resolution, the committee summary, and role denials (resident → help-desk API = 403, unknown phone = 404). All 17 passed.

11. Assumptions

Each assumption is labelled; I picked the interpretation that keeps the money and safety semantics of the brief intact.

- **A1 — Late fee.** The brief says late fees follow the society bye-laws but doesn't state a rate. I implemented **1.5% of the base amount per full month overdue**, stored on the bill, as a bye-law placeholder; the seed's numbers (e.g. Swati's October bill: ₹6,370 + ₹96 late fee) follow from it.
- **A2 — Seed scale.** The seed represents **24 flats across 5 towers of the 2,400 flats / 28 towers**, including the brief's three sample rows exactly (Neeraj T07-1203 paid; Swati T12-0408, tenant, 47 days overdue with a water-leak complaint; Aamir T21-1601 with ₹5,000 of ₹10,660 paid). Enough to make every screen, bucket and tier real; the engines have no scale assumptions.
- **A3 — Mocked externals.** OTP delivery, the payment gateway and the bank feed are mocked (test mode) exactly as the brief's core build instructs; the bank feed arrives as a CSV upload.
- **A4 — Resident UPI ids.** Each flat's seed carries the resident's UPI id as matching evidence (tiers 2–3). In production this is captured when a resident first pays in-app or registers their UPI id; flats without one simply can't match on tiers 2–3 and fall to the narration/queue paths.
- **A5 — Pinned reference date.** The seed pins “today” (26 Nov 2026) so ages, overdue buckets and the 47-days-overdue sample row are reproducible whenever the seed is re-run.

- **A6 — Committee is a role.** The core build names two roles (resident, help-desk) but requires a committee view; I gave the committee its own read-only login rather than folding the view into the help-desk account, because the committee is a different principal (the federation, not the facility company).

12. What I cut, and what is next

Cut to keep the core slice genuinely finished rather than broad and hollow:

- **Visitor & gate management** (guard tablet entry with photo, resident approvals, pre-approved guest codes) — the largest remaining module. Next up, and designed-for: a `Visitor` model (flat, guest, photo, status, timestamps) with the guard UI in the same minimal language but extra-large touch targets, and the DPDP rules built in from day one — photos purged after 90 days by a scheduled job, a privacy notice at the gates, offline-first entry with local queue + sync for network drops at the gate.
- **Amenity booking** (clubhouse/courts, slots, rules, fees) — a well-bounded module once the ledger exists to take its fees.
- **Family/tenant member management UI** — the `Membership` model already carries OWNER/TENANT/FAMILY; only the management screens are missing.
- **Resident rating on closure in the UI** — the field, storage and display exist; the tap-to-rate interaction on CLOSED complaints is a small follow-up.
- **Marathi and Hindi localisation.** Strings are centralised enough in components to extract; the brief's three-language requirement belongs with the visitor module rollout to guards and staff.
- **Real integrations:** SMS OTP (replace the console mock behind the same interface), a real payment gateway (replace the mock sheet; the receipt contract already matches a gateway callback), bank-feed API instead of CSV upload.
- **Postgres + deployment hardening** before real rollout: provider swap, connection pooling, and moving uploads off the service filesystem to object storage.
- **LLM classifier upgrade** behind the existing `classifyComplaint` contract, evaluated against the stored `aiConfidence` + human corrections this build already collects — the audit data to judge it by is a by-product of the v1 design.

The code, commit history and this document are the whole submission; `deployment.md` in the repo root covers running it, the demo logins and a 5-minute reviewer walkthrough.